Ruleta Rusa

Curso / Asignatura:

1º Desarrollo de Aplicaciones Web

Autor:

Alexandru Tofan Tofan

Repositorio GitHub:

GitHub Ruleta Rusa

Contents

DESCRIPCIÓN GENERAL DEL PROYECTO 3
REGLAS DEL JUEGO Y MECÁNICAS 3
- Número de participantes: 3
- Vidas iniciales: 3
- Elección de turno inicial: 3
- Tipos de balas: 3
- Selección de objetivo: 3
- Pasar turno: 4
- Penalización por dudar (Entrada no válida): 4
- Eliminación: 4
- Final de la partida: 4

DESARROLLO DE LA PARTIDA Y GESTIÓN DE TURNOS 4
Configuración inicial: 4
Bucle de la partida: 4
Control de turnos: 5
CONDICIONES DE VICTORIA Y ELIMINACIÓN 5
- Eliminación: 5
- Victoria: 5

DISEÑO TÉCNICO E IMPLEMENTACIÓN 5
5.1. Variables del Sistema 5
5.2. Estructuras de Control 7
5.3. Estructura de Datos y Arrays Paralelos 7
EMPLEO DEL RANDOM 7
1. Elegir quién empieza: 8
2. Determinar la bala: 8

PLAN DE PRUEBAS FUNCIONALES 8
DIFICULTADES TÉCNICAS Y SOLUCIONES 10
Error de índice fuera de rango (ArrayIndexOutOfBoundsException): 10
Doble selección aleatoria al empezar: 11
Cierre de programa al meter letras por consola: 11
RECURSOS EXTERNOS E INTELIGENCIA ARTIFICIAL 11
- Apuntes de clase y vídeos de YouTube: 11
- Inteligencia Artificial 11

PROPUESTAS DE MEJORA Y AMPLIACIÓN 11
9.1. Mejoras Técnicas (Organización del código) 12
9.2. Ampliaciones Funcionales (Mecánicas de juego) 12

DESCRIPCIÓN GENERAL DEL PROYECTO

Este proyecto es un videojuego desarrollado en Java para la consola de comandos, tomando como referencia la idea principal del juego *Buckshot Roulette*. La premisa consiste en sentar a una mesa a varios participantes (de 2 a 4 jugadores) que se van turnando para utilizar una pistola cargada con una mezcla aleatoria de balas reales y de fogueo.

Cada jugador empieza la partida con un total de 3 vidas. A medida que avanza la ronda, los participantes deben decidir si arriesgarse a disparar a un oponente, dispararse a sí mismos o pasar el turno. El objetivo final del juego es saber gestionar ese riesgo y ser el único jugador en pie cuando todos los demás hayan perdido sus vidas.

REGLAS DEL JUEGO Y MECÁNICAS

- **Número de participantes:** Se permite un mínimo de 2 y un máximo de 4 jugadores. Si se introduce un número diferente, el programa lo rechaza y vuelve a pedir la cantidad.
- **Vidas iniciales:** Todos los participantes inician la partida con 3 vidas.
- **Elección de turno inicial:** Se determina al azar usando la clase Random de Java justo al comenzar la partida.
- **Tipos de balas:**
    - **Bala Real:** Resta 1 vida al jugador al que se ha apuntado.
    - **Bala de Fogueo:** No resta vidas a ningún jugador; solo suena el clic del gatillo.

- **Selección de objetivo:** El jugador en turno puede elegir disparar a cualquiera de sus compañeros o bien arriesgarse y dispararse a sí mismo.

- **Pasar turno:** El jugador puede decidir saltar su turno de disparo y ceder la acción al siguiente.
- **Penalización por dudar (Entrada no válida):** Si el jugador mete un número distinto a las opciones mostradas en el menú de acciones, el programa muestra un aviso de error y el jugador pierde su turno por dudar.
- **Eliminación:** En cuanto el recuento de vidas de un jugador llega a 0, queda automáticamente eliminado de la partida. El sistema deja de asignarle turnos y ya no se le puede seleccionar como objetivo.
- **Final de la partida:** El juego se detiene cuando solamente queda un único jugador con vidas.

DESARROLLO DE LA PARTIDA Y GESTIÓN DE TURNOS

El flujo de la partida se divide en tres fases ordenadas:

Configuración inicial:

- El programa pide el número de jugadores (validando que sea entre 2 y 4).
- Mediante un bucle se van pidiendo e introduciendo los nombres de cada uno.
- Se asignan 3 vidas iniciales a cada posición correspondiente.
- Se genera el número aleatorio para decidir qué jugador empieza.

Bucle de la partida:

- Se muestra en pantalla de quién es el turno y cuántas vidas le quedan a cada participante.
- Se muestra el menú con las dos opciones posibles:
    1. *Tomar pistola y tentar a la suerte* (apuntar y disparar).
    2. *Pasar turno*.

- Si elige disparar, se saca la lista de jugadores disponibles, se escoge objetivo y el sistema saca el resultado de la bala (real o fogueo) para actualizar las vidas.

Control de turnos:

- Tras la acción o penalización, el turno avanza al siguiente sumando 1 a la posición del array (turnoActual++).
- Si el turno supera la cantidad total de jugadores, se resetea a 0 para que la rotación continúe en círculo.
- Si el jugador al que le tocaría el turno tiene 0 vidas (está eliminado), el sistema se salta esa posición y pasa directamente al siguiente jugador vivo.

CONDICIONES DE VICTORIA Y ELIMINACIÓN

- **Eliminación:** Se produce en el momento en que un participante recibe un total de 3 disparos con balas reales. El programa indica por consola que el jugador ha sido eliminado y deja su contador de vidas a 0.
- **Victoria:** Después de cada disparo que resta vidas, se hace un recuento comprobando cuántos participantes mantienen sus vidas por encima de 0. En el instante en que ese contador nos devuelve que solo queda 1 jugador con vidas, se detiene el bucle principal de la partida y se muestra el mensaje de victoria anunciando al ganador.

DISEÑO TÉCNICO E IMPLEMENTACIÓN

5.1. Variables del Sistema

A continuación, se detallan las variables principales utilizadas en el programa para controlar todo el funcionamiento de la partida:

Nombre de Variable	**Tipo de Dato**	**Valor Inicial**	**Propósito / Descripción**
totalJugadores	int	Introducido por usuario	Guarda cuántos jugadores van a participar (entre 2 y 4).
turnoActual	int	Aleatorio (0 a length-1)	Número que marca el índice del jugador que tiene el turno actual.
jugadorElegido	String	jugadores[turnoActual]	Variable auxiliar para mostrar por pantalla el nombre del jugador en turno.
opcionMenu	int	Introducido por usuario	Almacena la elección del menú de acciones (1 = Disparar, 2 = Pasar).
disparar	int	Introducido por usuario	Índice que guarda la posición del jugador al que se va a disparar.
aleatorioBala	int	Aleatorio (0 o 1)	Determina mediante Random si sale bala de fogueo o real.
esBalaReal	boolean	Según aleatorioBala == 1	Marca en true si la bala quita vida y en false si es fogueo.
partidaActiva	boolean	true	Mantiene el bucle while en marcha mientras quede más de un vivo.

5.2. Estructuras de Control

- **Bucle for:** Lo utilizamos principalmente al principio para repetir la recogida de nombres según los jugadores indicados y para asignar las 3 vidas iniciales a cada posición.
- **Bucle while:** Es el bucle central de la partida. Se mantiene ejecutando todo el proceso de turnos mientras la condición del boolean mantenga la partida en pie.
- **Condicionales if / else:** Los usamos para evaluar las opciones elegidas en los menús, controlar si una opción no es válida para restar el turno, verificar si un objetivo seleccionado aún tiene vidas y restar las vidas correspondientes.

5.3. Estructura de Datos y Arrays Paralelos

Para llevar el control de los datos del videojuego no se han usado clases complejas, sino dos **arrays paralelos**. Se llaman paralelos porque ambos se crean con el mismo tamaño (totalJugadores) y comparten la misma posición o índice:

- jugadores: Almacena los nombres de los participantes (String[]).
- vidasJugadores: Almacena el número de vidas que le quedan a cada uno (int[]).

De esta forma, la posición 0 de jugadores corresponde al nombre del Jugador 1, y la posición 0 de vidasJugadores contiene sus vidas en todo momento.

```
// Creamos los arrays de jugadores y sus respecti
String[] jugadores = new String[totalJugadores];/
int[] vidasJugadores = new int[totalJugadores];//
Arrays.fill(vidasJugadores, TOTALVIDAS); // Esto
```

Fragmento real del código en el que creamos los Arrays y declaramos las vidas con una constante

EMPLEO DEL RANDOM

La clase java.util.Random se utiliza en dos aspectos clave del juego:

1. **Elegir quién empieza:** Usamos random.nextInt(jugadores.length) para sacar una posición al azar entre 0 y totalJugadores - 1. Ese número se asigna a turnoActual.
2. **Determinar la bala:** Al apretar el gatillo hacemos random.nextInt(2). Esto genera solo dos números posibles:
    - 0: Bala de fogueo (50% de probabilidades).
    - 1: Bala real (50% de probabilidades).

PLAN DE PRUEBAS FUNCIONALES

Tabla detallada con los casos de prueba realizados para comprobar la estabilidad del programa frente a acciones válidas y entradas incorrectas:

Caso de Prueba	**Entrada**	**Resultado Esperado**	**Resultado Obtenido**	**Estado**
Iniciar partida con 2 jugadores	2	Se inicia la partida solicitando 2 nombres.	Pide los 2 nombres y empieza la partida.	**SUPERADA**
Iniciar partida con 4 jugadores	4	Se inicia la partida solicitando 4 nombres.	Pide los 4 nombres y empieza la partida.	**SUPERADA**
Límite de jugadores fuera de rango	1 o 5	Muestra mensaje de error y vuelve a solicitarlo.	Rechaza la entrada y pide el dato de nuevo.	**SUPERADA**

Caso de Prueba	Entrada	Resultado Esperado	Resultado Obtenido	Estado
Introducir texto en el número de jugadores	abc	Controla la excepción y no rompe la aplicación.	Muestra aviso sin cerrarse bruscamente.	**SUPERADA**
Disparar una bala real	Opción 1	El objetivo seleccionado pierde 1 vida.	Se resta 1 vida en la posición del array.	**SUPERADA**
Disparar una bala de fogueo	Opción 1	El objetivo conserva todas sus vidas.	Muestra el mensaje de salvado y mantiene vidas.	**SUPERADA**
Dispararse a uno mismo	Seleccionar propio número	Se le aplica el resultado de la bala a él mismo.	Procesa el disparo sobre el propio jugador.	**SUPERADA**
Pasar turno	Opción 2	Salta el turno sin realizar disparos.	Pasa la acción al siguiente participante.	**SUPERADA**
Opción de menú no válida	3 o 9	Pierde el turno por dudar como penalización.	Muestra el error y pasa el turno.	**SUPERADA**

Caso de Prueba	Entrada	Resultado Esperado	Resultado Obtenido	Estado
Disparar a un jugador eliminado	Índice de un jugador con 0 vidas	Muestra error y vuelve a pedir un objetivo válido.	Avisa de que está eliminado y no le dispara.	**SUPERADA**
Eliminación de un jugador	Vidas caen a 0	Notifica la eliminación y lo salta en los turnos.	Queda fuera de la rotación de turnos.	**SUPERADA**
Quedar un único jugador con vida	1 solo jugador vivo	Finaliza la partida y anuncia al ganador.	Sale del bucle while y muestra la victoria.	**SUPERADA**

DIFICULTADES TÉCNICAS Y SOLUCIONES

Durante el desarrollo del código surgieron algunos fallos concretos que hubo que corregir:

Error de índice fuera de rango (ArrayIndexOutOfBoundsException):

- *Problema:* Si jugaban 2 personas e introducías como objetivo el número 3, el programa intentaba buscar la posición 2 del array vidasJugadores, haciendo que el programa petara y se cerrara la consola.
- *Solución:* Añadimos una validación if (disparar >= 0 && disparar < jugadores.length) comprobando que el número metido esté dentro del rango del array antes de acceder a sus datos.

Doble selección aleatoria al empezar:

- *Problema:* Al principio se creaban dos variables aleatorias (indiceAleatorio y turnoActual), lo que hacía que una sobrescribiera a la otra y la primera selección no sirviera de nada.
- *Solución:* Se limpió el código para dejar una única llamada a random.nextInt(jugadores.length) guardada en turnoActual.

Cierre de programa al meter letras por consola:

- *Problema:* Cuando Scanner esperaba un número entero y el usuario ponía texto como abc, la aplicación saltaba con un error de tipo InputMismatchException.
- *Solución:* Se revisó la lectura en consola comprobando antes la entrada del Scanner para evitar que la partida finalizara bruscamente.

RECURSOS EXTERNOS E INTELIGENCIA ARTIFICIAL

Durante la realización del proyecto se han empleado diferentes apoyos de aprendizaje:

- **Apuntes de clase y vídeos de YouTube:** Consultados para entender mejor el funcionamiento de la clase Random y la forma de hacer que los turnos vuelvan al principio del array sin salirse de los límites cuando se llega al final.
- **Inteligencia Artificial:** Utilizada de forma puntual como herramienta de ayuda cuando los bucles y condicionales se atascaban (por ejemplo, al estructurar cómo saltarse los turnos de los jugadores eliminados), ayudando a ver en qué parte fallaba la lógica y comprendiendo el porqué de cada instrucción.

PROPUESTAS DE MEJORA Y AMPLIACIÓN

Dividimos las mejoras planteadas entre la organización interna del código y el desarrollo de nuevas mecánicas para próximas entregas:

9.1. Mejoras Técnicas (Organización del código)

- **Modularización:** Separar el código agrupado en el main dividiéndolo en métodos o funciones (por ejemplo, un método para pedir los datos iniciales, otro para mostrar los turnos y otro para gestionar los disparos).
- **Uso de switch:** Sustituir los condicionales if-else del menú principal por estructuras switch, permitiendo un código más claro y ordenado.

9.2. Ampliaciones Funcionales (Mecánicas de juego)

- **Menú de reinicio:** Permitir al usuario volver a jugar una nueva partida sin tener que ejecutar de nuevo la aplicación desde la consola.